



BRAINWARE UNIVERSITY

PYTHON PROJECT-BASED LEARNING REPORT (WEEK 1)

1. Basic Information

Title of the Project:	Student Grade Management System (Terminal-Based)
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS

2. Introduction and Background

Student grade management is a common academic administration task that involves recording, calculating, and retrieving student performance data. This project explores building a lightweight, terminal-based application that allows a user to manage student marks across a fixed set of subjects without relying on an external database or file storage, keeping all records in memory during a session.

Week 1 focuses on developing the core of the console application: defining the in-memory data model, building the grade-calculation logic, designing a bordered text-based user interface, and implementing a menu-driven loop that supports adding, viewing, searching, updating, and deleting student records. These form the base functionality on which further validation, persistence, or reporting features can be built in later weeks.

3. Objectives of the Project (Week 1)

- Design a menu-driven, terminal-based application for managing student academic records
- Implement functionality to add, view, search, update, and delete student grade records
- Develop a grade-calculation module that maps percentage scores to letter grades
- Build a reusable, bordered text UI (box-drawing utilities) for a cleaner terminal experience
- Validate user input, including roll-number uniqueness and marks range (0-100)
- Test the core CRUD (Create, Read, Update, Delete) operations for correctness

4. Problem Statement / Driving Question



How can student academic records - including subject-wise marks, totals, percentages, and grades - be reliably stored, calculated, and managed through a simple, dependency-free terminal application?

5. Methodology (Week 1 Activities)

The following tasks were completed during Week 1:

1. Data Model & Storage:

- Defined an in-memory STUDENTS dictionary keyed by roll number, storing each student's name and subject-wise marks
- Defined a fixed SUBJECTS list (Python, DBMS, COA) used consistently across all operations

2. Grade Calculation Logic:

- Implemented `calculate_grade()` to map a percentage score to a letter grade (A+ through F) using threshold checks
- Implemented `get_report()` to compute total marks, percentage, and grade for a given student
- Implemented `format_report()` to lay out a student's full report as display-ready lines

3. Terminal UI & Menu System:

- Built reusable box-drawing helpers (`box_top`, `box_bottom`, `box_divider`, `box_line`, `print_boxed`) for a bordered console layout
- Implemented a `main()` menu loop with six options: Add, View All, Search, Update, Delete, and Exit
- Added input validation, including duplicate roll-number checks and marks-range/type validation with `try/except`

6. Expected Outcomes / Deliverables (Week 1)

- A working console application supporting Add, View, Search, Update, and Delete of student grade records
- A functioning `calculate_grade()` module producing consistent letter grades from percentage scores
- A bordered, readable terminal UI built from reusable box-drawing utilities
- Input validation for duplicate roll numbers, invalid marks, and non-numeric entries
- Week 1 project report



7. Core Logic

The core of the program is the grade-calculation logic, which converts a student's overall percentage into a letter grade, and the reporting logic, which aggregates a student's marks into a total, percentage, and grade. Both are shown below with the surrounding threshold and aggregation logic that drives the rest of the menu options.

```
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B+"
    elif percentage >= 60:
        return "B"
    elif percentage >= 50:
        return "C"
    elif percentage >= 40:
        return "D"
    else:
        return "F"

def get_report(roll):
    student = STUDENTS[roll]
    total = sum(student["marks"].values())
    max_total = 100 * len(SUBJECTS)
    percentage = (total / max_total) * 100
    grade = calculate_grade(percentage)
    return student, total, percentage, grade
```

`calculate_grade()` checks the percentage against descending thresholds (90, 80, 70, 60, 50, 40) and returns the corresponding grade, defaulting to "F" below 40. `get_report()` sums the marks dictionary, derives the percentage against the maximum possible total, and calls `calculate_grade()` to attach a grade - this single function backs the View, Search, and Update-confirmation flows in the menu.



8. Output Screenshot

```
STUDENT  
GRADE  
MANAGEMENT  
SYSTEM  
  
Terminal Student Grade Management System  
  
|| [1] Add Student  
|| [2] View All Students  
|| [3] Search Student  
|| [4] Update Marks  
|| [5] Delete Student  
|| [6] Exit  
||  
> Enter option: 1  
> Enter roll number: 002  
> Enter student name: nam  
> Enter marks in Python (out of 100): 55  
> Enter marks in DBMS (out of 100): 55  
> Enter marks in COA (out of 100): 55  
  
|| STUDENT ADDED :  
||  
|| Roll no : 002  
|| Name : nam  
||  
  
|| [1] Add Student  
|| [2] View All Students  
|| [3] Search Student  
|| [4] Update Marks  
|| [5] Delete Student  
|| [6] Exit  
||  
> Enter option: 
```



9. Project Timeline

Week	Date	Activity	Status
Week 1	10/07/2026	Student Grade Management System	Current Week
Week 2	17/07/2026	Program 1: Electricity Bill Calculator	Upcoming
Week 2	17/07/2026	Program 2: ATM Transaction Simulator	Upcoming
Week 3	24/07/2026	Program 1: Bank Management System	Upcoming
Week 3	24/07/2026	Program 2: Hotel Booking System	Upcoming
Week 3	24/07/2026	Program 3: Library Management System	Upcoming
Week 4	31/07/2026	Program 1: Password Strength Checker	Upcoming
Week 4	31/07/2026	Program 2: Email Validation System	Upcoming
Week 5	TBD	Text Encryption & Decryption Tool	Upcoming

10. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Primary programming language used to implement the entire console application
Built-in Data Structures (dict)	In-memory storage of student records via the STUDENTS dictionary
Python Standard I/O (input/print)	Handling menu-driven user interaction in the terminal
Exception Handling (try/except)	Validating numeric marks input and catching ValueError
ASCII/Unicode Box-Drawing Characters	Building the bordered terminal UI (box_top, box_bottom, box_line, etc.)

11. References

- Python 3 Official Documentation: <https://docs.python.org/3/>
- Python Official Docs - Dictionaries: <https://docs.python.org/3/tutorial/datastructures.html#dictionaries>
- Python Official Docs - Errors and Exceptions: <https://docs.python.org/3/tutorial/errors.html>
- Unicode Box Drawing Characters Reference: https://en.wikipedia.org/wiki/Box-drawing_character
- Brainware University - Machine Learning Course Guidelines



BRAINWARE UNIVERSITY

12. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
